

Failure handling (three errors, three reactions) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Where this exercise is saved: `practice/custom_errors_practice.py`, under its `# Failure handling` section — the same file also holds the [Basic exercise](#), under a `# Basic` section. Run it with `cd practice && python custom_errors_practice.py`.

Builds on: the `# Basic` section of that same file — same habit, three error classes instead of one. Nothing to copy or import: scroll up in the file you already have.

Story — `custom_errors_practice.py` (Failure handling section): a real program has more than one way to fail, and each way usually needs a different reaction — retry, ask for new payment info, ask for a new address. Three separate error types and three separate `except` blocks make that difference explicit in the code itself. **If not:** a single `except Exception` would treat "out of stock" the same as "payment declined," so the code would end up with an `if/elif` chain re-deriving what the exception type already told it for free.

Basic Version

Approach 1 — three separate `except` blocks, each reacting differently

```
# practice/custom_errors_practice.py – Failure handling section
class OutOfStockError(Exception):
    pass

class PaymentDeclinedError(Exception):
    pass

class InvalidAddressError(Exception):
    pass

def process_order(problem_type):
    if problem_type == "stock":
        raise OutOfStockError("item is out of stock")
    if problem_type == "payment":
        raise PaymentDeclinedError("payment was declined")
```

```
if problem_type == "address":
    raise InvalidAddressError("shipping address is invalid")

for problem_type in ["stock", "payment", "address"]:
    try:
        process_order(problem_type)
    except OutOfStockError as e:
        print("Will retry later, telling the warehouse: " + str(e))
    except PaymentDeclinedError as e:
        print("Asking for a different payment method: " + str(e))
    except InvalidAddressError as e:
        print("Asking the customer to fix their address: " + str(e))
```

Expected output:

```
Will retry later, telling the warehouse: item is out of stock
Asking for a different payment method: payment was declined
Asking the customer to fix their address: shipping address is invalid
```

This is correct and is exactly what the exercise asks for: 3 error types, each with its own reaction. It's missing type hints, which is fine for a first working version.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — three separate, differently-reacting `except` blocks

```
# practice/custom_errors_practice.py – Failure handling section
class OutOfStockError(Exception):
    pass

class PaymentDeclinedError(Exception):
    pass

class InvalidAddressError(Exception):
    pass

def process_order(problem_type: str) -> None:
    # why: simulates three different real failures on demand, so the
    # except blocks below have something real to react to.
    # when: in real code this would be wherever an order actually gets
    # processed – the type of failure comes from what really went wrong, not a
    # parameter.
    if problem_type == "stock":
        raise OutOfStockError("item is out of stock")
    if problem_type == "payment":
        raise PaymentDeclinedError("payment was declined")
    if problem_type == "address":
        raise InvalidAddressError("shipping address is invalid")

for problem_type in ["stock", "payment", "address"]:
    try:
        process_order(problem_type)
    except OutOfStockError as e:
        # how: Python checks except blocks top to bottom and stops at the
        # first type match – each error type here gets its own, different
        # reaction.
        print(f"Will retry later, notifying the warehouse: {e}")
    except PaymentDeclinedError as e:
        print(f"Asking the customer for a different payment method: {e}")
```

```
except InvalidAddressError as e:
    print(f"Asking the customer to fix their address: {e}")
```

Expected output:

```
Will retry later, notifying the warehouse: item is out of stock
Asking the customer for a different payment method: payment was declined
Asking the customer to fix their address: shipping address is invalid
```

Approach 2 — a shared base class, for when some handling really is shared

```
# practice/custom_errors_practice.py – Failure handling section
class OrderError(Exception):
    pass

class OutOfStockError(OrderError):
    pass

class PaymentDeclinedError(OrderError):
    pass

class InvalidAddressError(OrderError):
    pass

def process_order(problem_type: str) -> None:
    if problem_type == "stock":
        raise OutOfStockError("item is out of stock")
    if problem_type == "payment":
        raise PaymentDeclinedError("payment was declined")
    if problem_type == "address":
        raise InvalidAddressError("shipping address is invalid")

for problem_type in ["stock", "payment", "address"]:
    try:
        process_order(problem_type)
    except PaymentDeclinedError as e:
        print(f"Asking the customer for a different payment method: {e}")
    except OrderError as e:
```

```
# catches OutOfStockError and InvalidAddressError together
print(f"Order problem, logging and notifying support: {e}")
```

Expected output:

```
Order problem, logging and notifying support: item is out of stock
Asking the customer for a different payment method: payment was declined
Order problem, logging and notifying support: shipping address is invalid
```

Difference from Basic: both Intermediate approaches add full type hints (`problem_type: str, - > None`). The real difference is between the two approaches: Approach 1 reacts to all 3 errors independently. Approach 2 makes `OutOfStockError` and `InvalidAddressError` inherit from a shared `OrderError` base and handles them together — notice the output changes for the stock and address cases, because they now hit the shared `except OrderError:` branch instead of their own. `PaymentDeclinedError` is still caught on its own, since Python checks `except` clauses top to bottom and stops at the first match.

Which one should you actually write? Intermediate Approach 1 (three separate `except` blocks, each reacting differently) is what you should write — it's exactly what's being tested, and it's the clearest code for 3 error types. Intermediate Approach 2's shared base class is worth it the moment two or more error types really do deserve the same reaction.